

Observability, Security & Cost

By Day 9 the system is too complex to debug by tail-ing CloudWatch. Today: 3 pillars, EMF, X-Ray across Step Functions + Lambda + Bedrock + SageMaker, IAM tightening, cost as observability. The OBSERVABILITY CONTEXT block in .windsurfrules v7.

Day 9 → Day 10

Yesterday the wrapper Lambda joined; ASL Parallel forks Bedrock + your-model. Today the system gets EYES — logs, metrics, traces, alarms, and cost meters — across all 10 Lambdas + Bedrock + SageMaker + Step Functions.

YESTERDAY

Wrapper Lambda + ASL Parallel + Choice

↓ needs ↓

TODAY

X-Ray cross-service trace through SFN→Lambda→Bedrock→SageMaker. One correlation_id per record across all 10 Lambdas.

YESTERDAY

ModelLatency + ModelSetupTime in CW

↓ joined by ↓

TODAY

AWS/Bedrock InvokeModel latency / throttle + AWS/SageMaker overhead — first-class dashboard widgets.

YESTERDAY

.windsurfrules v6 (7 blocks), 8 HITL dims

↓ +1 ↓

TODAY

OBSERVABILITY CONTEXT block (.windsurfrules → v7) (correlation_id propagation, EMF metric namespace, log retention, trace sampling).

YESTERDAY

PipelineRole with permissive S3/SFN/Lambda

↓ tightened ↓

TODAY

Wildcard audit. ARN-scoped Resource. Conditions on s3:prefix and stepfunctions:StateMachineArn.

YESTERDAY

Sandbox: 10 of 10 Lambdas (final cap from D9); ~43/50 Bedrock

↓ unchanged ↓

TODAY

0 new Lambdas today (instrumentation only). ~5 Bedrock for Cascade-authoring of dashboard JSON.

YESTERDAY

Spends inferred from console

↓ measured ↓

TODAY

CW metric proxies (AWS/Lambda Duration × Invocations + AWS/Bedrock token counts + AWS/Athena DataScannedInBytes) per env=lab, learner=\$LEARNER tag. Cost Explorer is taught conceptually but NOT used in this sandbox (CE API not enabled).

Why observability now — 10 Lambdas + 1 ASL + 4 services > one log tail

THE PROBLEM — A FAILED RECORD CAN HIDE IN 12 PLACES

By D9 a single record traverses producer Lambda → Kinesis → consumer Lambda → SFN start → Bedrock branch (or its retry) + wrapper Lambda → SageMaker (cold or hot) → merge Lambda → S3/SQS sinks. If it goes missing, tail-ing one log group tells you nothing. You need correlation IDs that travel WITH the record + dashboards that answer 4 questions: flowing? correct? fast? cheap?

THE SCOPE — 3 PILLARS + COST AS A FOURTH

Logs (powertools Logger + correlation_id), metrics (EMF custom + AWS/Bedrock + AWS/SageMaker + AWS/States), traces (X-Ray cross-service), and cost (CW metric proxies — Cost Explorer API is taught conceptually but disabled in the Pluralsight sandbox so live evidence comes from CW). Add IAM tightening and per-resource log-retention as security/cost hygiene. Sandbox-respecting: 0 new Lambdas — instrumentation only.

By the end of Day 10 you can...

INSTRUMENT

Add aws-lambda-powertools Logger + correlation_id propagation across all 10 Lambdas. Verify X-Ray cross-service trace shows SFN→Lambda→Bedrock→SageMaker as one trace.

ALARM

Configure 4 SNS alarms: IteratorAge>30s, AWS/Bedrock ThrottlingException>0/5min, confidence-distribution drift (mean drop >20%), DailyCost>\$5/learner.

EMIT

Emit 4 EMF custom metrics (records_processed, low_confidence_count, throttle_retries, model_setup_ms) from the wrapper Lambda. Verify they appear under namespace AccelyaPipeline/<Learner>.

TIGHTEN

Audit accelya-pipeline-role-<learner> — replace s3:* with explicit verbs; add Conditions on s3:prefix=enriched/<learner>/*; scope stepfunctions:StartExecution to one ARN.

DASHBOARD

Cascade-author a CloudWatch dashboard JSON answering FOUR questions (flowing? correct? fast? cheap?) with widgets from EMF, AWS/Bedrock, AWS/SageMaker, AWS/States.

MEASURE

Compute cost from CW metric proxies (AWS/Lambda Duration × Invocations + AWS/Bedrock InputTokenCount + OutputTokenCount + AWS/Athena DataScannedInBytes) tagged env=lab + learner=\$LEARNER; identify top 3 cost drivers; document in /reviews/m10-cost-baseline.md. Cost Explorer API is taught as the production pattern but NOT executed in this sandbox.

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Seven concepts. 3 pillars. EMF custom metrics. X-Ray cross-service. AWS/Bedrock + AWS/SageMaker namespaces. Correlation IDs. The OBSERVABILITY CONTEXT block in .windsurfrules v7.

TIME

~30 min

02

Guided design

Four guided slides. Dashboard answering 4 questions. IAM tightening pass. Alarm definitions. Cost as observability via CW metric proxies + per-tag (Cost Explorer taught conceptually, not used in sandbox).

TIME

~35 min

03

Three labs

Lab 1 — powertools Logger + correlation IDs across 10 Lambdas; verify X-Ray (45 min). Lab 2 — dashboard JSON + EMF + 4 SNS alarms (45 min). Lab 3 — IAM tightening MR + cost baseline (45 min).

TIME

~135 min

04

Review & wrap

Five Day-10 mistakes. End-of-day cleanup. Day-10 in numbers. Bridge to Day 11 — End-to-End Integration WITH CI/CD inside. The DEPLOY CONTEXT block in .windsurfrules v8.

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Seven concepts. The three pillars (logs/metrics/traces). EMF for Lambda-friendly custom metrics. X-Ray cross-service traces. AWS/Bedrock + AWS/SageMaker first-class metrics. Correlation IDs via powertools. The OBSERVABILITY CONTEXT block in .windsurfrules v7.

Logs answer WHAT. Metrics answer HOW MANY / HOW FAST. Traces answer WHERE.

Each pillar has a different cost profile and a different answer it's good at giving. By Day 10 you need all three because no single pillar tells you why a record disappeared between SFN start and the merge step.

What each pillar is for, and what it costs in this pipeline

LOGS (CloudWatch Logs):

Best for: free-text context, request bodies, exception messages.

Cost driver: ingestion + retention. Default 'never expire' is the silent killer.

Discipline: powertools Logger + JSON + correlation_id; retention 14d in lab, 30d production.

METRICS (CloudWatch Metrics):

Best for: time-series counts/percentiles you want to alarm on.

Cost driver: per custom metric per month + GetMetricData calls.

Discipline: EMF (embedded metric format) — emit metric INSIDE the log line.

AWS auto-emits AWS/Lambda, AWS/Bedrock, AWS/SageMaker, AWS/States — use these first.

TRACES (X-Ray):

Best for: 'where did the time go?' across services in one request.

Cost driver: per-trace + per-service. Sample at 10% in production.

Discipline: enable X-Ray on Lambda + SFN + DynamoDB + Bedrock + SageMaker.

COST (the unofficial 4th pillar):

Best for: 'what is this pipeline costing me per record?'.

Cost driver: Cost Explorer API is free in production but NOT enabled in this sandbox; we use CW metric proxies (Lambda Duration × Invocations,

Bedrock token counts, Athena DataScannedInBytes) tagged env=lab + learner=\$LEARNER.

Discipline: per-tag cost report; daily \$5 alarm per learner.

01

Logs = WHAT

Free-text context. Default retention is the killer.

02

Metrics = HOW MANY

EMF embeds metric in log. Cheap; alarmable.

03

Traces = WHERE

X-Ray cross-service. Sample to control cost.

04

Cost = HOW MUCH

Tag everything; CW proxies in this sandbox (CE API not enabled); production swaps in Cost Explorer.

05

All four needed

By Day 10 no single pillar can locate a vanished record.

Embedded Metric Format: emit metric INSIDE a log line. Avoids synchronous PutMetricData.

EMF is the Lambda-friendly way to publish custom metrics — write a special JSON log line; CloudWatch parses it. Cheaper than PutMetricData (no API charge). NOT a timeout-survival guarantee: powertools.Metrics @log_metrics decorator flushes at handler completion or exception, but a HARD Lambda timeout terminates the process before flush. For timeout-risk paths, flush early/per-stage or emit critical counters BEFORE long calls.

EMF format + powertools Metrics helper

```
## EMF JSON shape (what CloudWatch parses out of a log line)
{
  "_aws": {
    "CloudWatchMetrics": [{
      "Namespace": "AccelyaPipeline/<learner>",
      "Dimensions": [{"Function", "Stage"}],
      "Metrics": [{"Name": "ModelSetupMs", "Unit": "Milliseconds"}]
    }],
    "Timestamp": 1731070800000
  },
  "Function": "accelya-predict-cancel-risk-<learner>",
  "Stage": "prod",
  "ModelSetupMs": 4127,
  "correlation_id": "corr-abc-123"
}

## With aws-lambda-powertools (lab uses this – no JSON crafting):
from aws_lambda_powertools import Logger, Metrics
from aws_lambda_powertools.metrics import MetricUnit
logger = Logger(service='accelya-predict')
metrics = Metrics(namespace='AccelyaPipeline/<learner>',
                  service='accelya-predict')

@metrics.log_metrics(capture_cold_start_metric=True)
@logger.inject_lambda_context(correlation_id_path='correlation_id')
def handler(event, context):
    metrics.add_metric(name='ModelSetupMs',
                      unit=MetricUnit.Milliseconds, value=4127)
    metrics.add_metric(name='LowConfidenceCount',
                      unit=MetricUnit.Count, value=1)
    logger.info('inference complete', extra={'risk_score': 0.42})
## Output: ONE structured log line per invocation = ONE metric publish.

## TIMEOUT-RISK PATTERN – flush early so critical counters land even on timeout:
def handler(event, context):
    metrics.add_metric(name='RecordsReceived', unit=MetricUnit.Count, value=1)
    metrics.flush_metrics() # explicit early flush BEFORE long call
    result = sm.invoke_endpoint(...) # long call; may exhaust Lambda timeout
    metrics.add_metric(name='RecordsCompleted', unit=MetricUnit.Count, value=1)
    # @log_metrics decorator flushes again on completion (idempotent)
## Hard timeout between flush and decorator-flush → only post-flush metric is lost,
## not the entire batch.
```

01

EMF = log-embedded metric

Avoids synchronous PutMetricData. No API charge.

02

powertools Metrics

Hides JSON crafting. Decorator-based.

03

Namespace per learner

AccelyaPipeline/<learner> isolates dashboards.

04

Capture cold-start metric

Decorator publishes ColdStart=1 on first invoke.

05

Hard timeout = lost flush

Decorator flushes on exit; HARD timeout kills first. For risky paths flush early.

One trace covers SFN execution → Lambda invokes → Bedrock InvokeModel → SageMaker invoke_endpoint → DDB.

X-Ray ties together the 5 services your record passes through. Without it, Bedrock latency vs SageMaker latency vs Lambda init time looks like one big number. With it, each segment shows independently with timing + status.

Enabling X-Ray + sampling rule + propagating trace ID

```
## ENABLE on Lambda - one CLI flag per function:
aws lambda update-function-configuration \
  --function-name accelya-consumer-$LEARNER \
  --tracing-config Mode=Active --region ap-south-1
# Repeat for all 10 Lambdas.

## ENABLE on Step Functions:
aws stepfunctions update-state-machine \
  --state-machine-arn $SM_ARN \
  --tracing-configuration enabled=true \
  --region ap-south-1

## ENABLE on Bedrock + SageMaker - automatic when called from
## a Lambda with Active tracing + AWSXRayDaemonWriteAccess on the role.
## boto3 1.34+ instruments these clients via aws-xray-sdk:
from aws_xray_sdk.core import patch_all
patch_all() # patches boto3, requests, urllib3 - call ONCE at import time

## CRITICAL - STITCH SFN INTO THE TRACE via traceHeader
## SFN tracing alone does NOT inherit the upstream Lambda's trace context.
## Consumer Lambda MUST extract its current X-Ray trace + pass as traceHeader
## on StartExecution, otherwise the SFN execution becomes a NEW root trace
## (and your service map shows two disconnected trees).
from aws_xray_sdk.core import xray_recorder
seg = xray_recorder.current_segment()
trace_header = f'Root={seg.trace_id};Parent={seg.id};Sampled=1'
sfn.start_execution(stateMachineArn=SM_ARN,
                    name=f'inf-{event_id}',
                    input=json.dumps(payload),
                    traceHeader=trace_header) # <-- stitches the trace

## IAM - TWO DIFFERENT POLICIES on TWO DIFFERENT ROLES
## Lambda role: AWSXRayDaemonWriteAccess (PutTraceSegments + PutTelemetryRecords)
## SFN role: xray:PutTraceSegments + xray:PutTelemetryRecords
## + xray:GetSamplingRules + xray:GetSamplingTargets
## (managed AWSXRayDaemonWriteAccess does NOT include the GetSampling*)

## SAMPLING RULE (lab - 100% to see everything; production drop to 10%):
{
  "RuleName": "accelya-pipeline-lab",
  "Priority": 1000,
  "FixedRate": 1.0, # 100% in lab
  "ReservoirSize": 1,
  "ServiceName": "accelya-*",
  "ServiceType": "*",
  "Host": "*",
  "HTTPMethod": "*",
  "URLPath": "*",
  "ResourceARN": "*"
}

## PROPAGATING correlation_id:
## Producer Lambda: correlation_id = f'corr-{uuid.uuid4()}' INTO the Kinesis record.
## Consumer Lambda: extract from record; pass into SFN input as $.correlation_id.
## All downstream Lambdas: powertools Logger inject lambda_context picks it up.
## X-Ray TraceId is independent - both travel; both visible in CloudWatch Insights.
```

01

1 trace, 5 services

SFN → Lambda → Bedrock → SageMaker → DDB.

02

patch_all() once

aws-xray-sdk auto-instruments boto3 clients.

03

Sample 10% in prod

Lab 100% to see everything; cost grows linearly.

04

correlation_id ≠ TraceId

Both travel; correlation_id is yours, X-Ray's is auto.

05

AWSXRayWriteOnlyAccess

Lambda role needs it; otherwise traces silently dropped.

Use what AWS publishes for free before adding custom metrics. Verified catalogue for THIS lab — not exhaustive across all model/endpoint types.

Both services emit rich metrics under their own namespace. Wire dashboards to these FIRST; only add EMF for things AWS doesn't measure (e.g., LowConfidenceCount, CostPerRecordEstimate).
 Note: SageMaker metrics depend on endpoint variant + lifecycle — D9 endpoint was DELETED at end of D9 wrap, so today's dashboard shows historical-only SageMaker data.

What's already published — by namespace (verified for this pipeline)

AWS/Bedrock (per ModelId dimension):

Invocations # call count
 InvocationLatency # round-trip ms; alarm at p99 > 8000ms
 InvocationClientErrors # 4xx (often throttle/auth)
 InvocationServerError # 5xx
 InvocationThrottles # ThrottlingException specifically — alarm at >0/5min
 InputTokenCount # cost driver (sum) — wire to daily SNS
 OutputTokenCount # cost driver (sum)

AWS/SageMaker (per EndpointName, VariantName dimension):

Invocations # call count
 ModelLatency # endpoint-internal latency (ms)
 OverheadLatency # AWS-internal (cold-start visible here)
 ModelSetupTime # cold-start specifically (Serverless only)
 Invocation4XXErrors / Invocation5XXErrors

AWS/Lambda (per FunctionName dimension):

Invocations / Errors / Duration / Throttles / IteratorAge (ESM-bound)
 ConcurrentExecutions # alarm at 80% of account quota
 ProvisionedConcurrencySpilloverInvocations # for D11

AWS/States (per StateMachineArn dimension):

ExecutionsStarted / Succeeded / Failed / TimedOut / Aborted
 ExecutionTime # wall-clock; alarm at p99 > target SLA

AWS/Kinesis (per StreamName dimension):

IncomingRecords / IteratorAgeMilliseconds (CRITICAL for ESM lag)
 WriteProvisionedThroughputExceeded (provisioned mode only)

RULE — only add a custom metric when no AWS/* covers it.

Day 10 lab adds 4: records_processed_per_correlation_id, low_confidence_count,

throttle_retries (across both Bedrock and SageMaker), confidence_value (histogram bucket).

01

AWS/* first

Free, no instrumentation. Custom only for the gap.

02

Bedrock InvocationThrottles

Alarm at >0 per 5min — first sign of cap pressure.

03

SageMaker ModelSetupTime

Serverless cold-start in real time. Tunable knob.

04

IteratorAgeMilliseconds

ESM falling behind = consumer underprovisioned.

05

TokenCount = cost

Sum daily; wire to SNS at projected \$5/day.

One ID travels with the record from producer to last sink. Lets CloudWatch Insights find every log line for one event.

powertools Logger has first-class correlation_id support. Producer assigns; every Lambda extracts and re-emits in every log line. CloudWatch Insights query then becomes a one-liner.

Generation, propagation, query — the full lifecycle

```
## STEP 1 — Producer assigns correlation_id and embeds in record
import uuid
correlation_id = f'corr-{uuid.uuid4()}'
envelope = {
    'event_id':      str(uuid.uuid4()),
    'event_type':    'cancellation',
    'event_version': 1,
    'correlation_id': correlation_id, # NEW — travels with record
    'data':          {...}
}
kinesis.put_record(StreamName=STREAM, Data=json.dumps(envelope),
                  PartitionKey=pk)

## STEP 2 — Consumer extracts + injects into Lambda context
## (powertools Logger picks up correlation_id_path automatically)
from aws_lambda_powertools import Logger
logger = Logger(service='accelya-consumer')

@logger.inject_lambda_context(correlation_id_path='envelope.correlation_id',
                             log_event=False)
def handler(event, context):
    # logger.info auto-includes correlation_id in EVERY log line
    logger.info('record received')
    sfm.start_execution(StateMachineArn=SM_ARN,
                       name=f'inf-{event["event_id"]}',
                       input=json.dumps(**event, 'correlation_id': correlation_id))

## STEP 3 — Downstream Lambdas (wrapper, merge) re-extract via the same
## decorator with correlation_id_path='correlation_id' (now top-level)
## FALLBACK ORDER (defensive — handles mixed-source events):
##   envelope.correlation_id → correlation_id → generated f'corr-{uuid.uuid4()}'
## Implement as a tiny extractor in src/common/observability.py so all
## handlers share the same fallback chain.

## STEP 4 — Find every log line for one record:
## CloudWatch Insights (across ALL Lambda log groups) — one query:
fields @timestamp, @message
| filter correlation_id = 'corr-abc-123'
| sort @timestamp asc
## Returns: producer → consumer → wrapper → merge → sink in one timeline.
```

01

Producer assigns

uuid-prefixed for grep-ability.

02

Travels in record + SFN input

Embedded in envelope; propagated via SFN \$.correlation_id.

03

powertools auto-injects

@inject_lambda_context with correlation_id_path.

04

Insights query is 3 lines

filter + sort. Every log line for one record.

05

Independent of X-Ray TraceId

Both travel; correlation_id is durable + grep-able.

Day-7→Day-9 role grew permissive. Today: replace s3:* / dynamodb:* / lambda:* with explicit verbs + ARN-scoped Resource + Conditions.

Three classic wildcards to remove: action wildcards (s3:*), resource wildcards (Resource:*), and missing Conditions (any prefix any time). Each one becomes a finding in a real SOC 2 audit. Lab does the audit + the MR.

BEFORE → AFTER — the three transformations

```
## BEFORE — what accelya-pipeline-role-<learner> looks like end of D9
{
  "Effect": "Allow",
  "Action": ["s3:*"],
  "Resource": "*"          # finding 1: resource wildcard
}

## AFTER — explicit verbs, scoped ARN, Conditions
{
  "Sid": "S3WriteEnrichedOnly",
  "Effect": "Allow",
  "Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
  "Resource": [
    "arn:aws:s3::accelya-airline-data-ap-south-1",
    "arn:aws:s3::accelya-airline-data-ap-south-1/enriched/<learner>/*",
    "arn:aws:s3::accelya-airline-data-ap-south-1/quarantine/<learner>/*"
  ],
  "Condition": {
    "StringLike": { "s3:prefix": ["enriched/<learner>/*",
                                "quarantine/<learner>/*"] },
    "Bool": { "aws:SecureTransport": "true" }
  }
}

## STEP-FUNCTIONS — scope to ONE state machine ARN
{
  "Action": ["states:StartExecution", "states:DescribeExecution"],
  "Resource": "arn:aws:states:ap-south-1:$ACCT:stateMachine:inference-pipeline-<learner>"
}

## BEDROCK — scope to ONE inference profile (the apac one we've used)
{
  "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
  "Resource": [
    "arn:aws:bedrock:ap-south-1:$ACCT:inference-profile/apac.anthropic.claude-3-5-sonnet-20241022-v2:0"
  ]
}

## SAGEMAKER — scope to ONE endpoint
{
  "Action": ["sagemaker:InvokeEndpoint"],
  "Resource": "arn:aws:sagemaker:ap-south-1:$ACCT:endpoint/accelya-cancel-risk-<learner>"
}

## VERIFICATION — IAM Access Analyzer policy validation
aws accessanalyzer validate-policy \
  --policy-document file://policy.json \
  --policy-type IDENTITY_POLICY --region ap-south-1
# Expected: "findings": [] — any GENERAL_WARNING or SECURITY_WARNING → fix
```

01**Three wildcards to kill**

Action, Resource, missing Conditions.

02**ARN-scope every line**

S3 prefix + SFN ARN + Bedrock profile + endpoint.

03**aws:SecureTransport=true**

Force HTTPS — silent SOC 2 finding otherwise.

04**Access Analyzer validates**

Run validate-policy; expect [] findings.

05**Open MR on GitLab**

Pair-reviewed; never apply directly to lab role.

Added to .windsurfrules v6 (becomes v7).

Cascade now reads OBSERVABILITY CONTEXT before scaffolding any new Lambda or refactoring an existing one. Tells it: correlation_id propagation rule, EMF namespace, log retention target, X-Ray sampling, alarm thresholds.

OBSERVABILITY CONTEXT layer template

```
## OBSERVABILITY CONTEXT
Logger:      aws-lambda-powertools.Logger(service=<short-name>)
Log retention: 14 days (lab) | 30 days (production)
Log format:   JSON (powertools default)
Correlation ID:
- Producer assigns: f'corr-{uuid.uuid4()}'
- Path in record:   envelope.correlation_id
- Path downstream:  correlation_id (top-level after consumer extraction)
- Decorator:        @logger.inject_lambda_context(correlation_id_path=...)

Metrics namespace: AccelyaPipeline/<learner>
EMF helper:        aws-lambda-powertools.Metrics
Cold-start metric: capture_cold_start_metric=True (decorator flag)
Custom metrics allowed:
- records_processed_per_correlation_id
- low_confidence_count
- throttle_retries
- confidence_value (for histogram)
Custom metrics FORBIDDEN if AWS/* covers it:
- Lambda Duration → use AWS/Lambda Duration
- Bedrock latency → use AWS/Bedrock InvocationLatency
- SageMaker cold-start → use AWS/SageMaker ModelSetupTime

X-Ray:          Active on all 10 Lambdas + SFN; aws_xray_sdk.patch_all()
Sampling rule:   accelya-pipeline-lab (100% lab, 10% production)

Alarms (3 CW + 1 cost monitor in lab):
IteratorAgeMilliseconds > 30000 → CW alarm → SNS accelya-ops-<learner>
AWS/Bedrock InvocationThrottles > 0/5min → CW alarm → SNS
Confidence drift via metric math (current 1h vs prev 1h, drop>20%) → CW alarm → SNS
Daily cost > $5/learner → CW metric-math alarm on cost-proxy expression (in-region, real-time);
Cost Anomaly Detection mentioned conceptually only (us-east-1 + lagged + CE API not enabled in sandbox)

Tags (mandatory on every resource):
env=lab
learner=$LEARNER
module=10

Forbidden:
- Logging request bodies that contain PII (booking_id is allowed; passenger_name no)
- Custom metric where AWS/* covers it (cost duplication)
- X-Ray on >10% sampling in production (cost runaway)
- Log retention default (never expire) — sets log group to 14d explicitly
```

01

Layer slot

Between MODEL CONTEXT and CATALOG CONTEXT.

02

powertools everywhere

Logger + Metrics + Tracer — single dependency.

03

AWS/* before custom

Forbidden list names the duplicates.

04

4 mandatory alarms

IteratorAge, Bedrock throttle, conf drift, daily cost.

05

Tags mandatory

env+learner+module — without these Cost Explorer is blind.

Observability vocabulary set

3 pillars + cost. EMF for custom metrics (NOT timeout-survival — flush early on risk paths). X-Ray for cross-service traces (with traceHeader on StartExecution). AWS/Bedrock + AWS/SageMaker first. Correlation IDs via powertools with fallback chain. IAM tightened from wildcards. The OBSERVABILITY CONTEXT block in .windsurfrules v7.

VOCABULARY

EMF, powertools, correlation_id (with fallback), X-Ray segment + traceHeader, IteratorAgeMilliseconds, ModelSetupTime, InvocationThrottles, Access Analyzer (gate on ERROR+SECURITY_WARNING).

WHAT'S NEW

OBSERVABILITY CONTEXT block in .windsurfrules v7. 4 EMF custom metrics + CostPerRecordEstimate. 3 CW alarms + 1 cost monitor. 1 IAM tightening MR (with safe-rollout). 0 new Lambdas.

WHAT'S NEXT

Four guided design slides: dashboard answering 4 questions, IAM tightening pass, 3 CW alarms (cost is its own slide), cost as observability via per-record EMF metric.

02

PART 2 · ~35 MINUTES

Guided design

Four guided design slides. Dashboard JSON answering 4 questions. IAM-tightening pass on accelya-pipeline-role-<learner>. Four alarm specs. Cost as observability via Cost Explorer + per-tag.

Single CloudWatch dashboard. Each row = one question. Cascade-author from JSON template.

DASHBOARD STRUCTURE

4 rows × 2-3 widgets each. Each row has a single-line question header.

```

ROW 1 — IS IT FLOWING?
  Widget 1.1: AWS/Kinesis IncomingRecords (line, 5min, 1h)
  Widget 1.2: AWS/Kinesis IteratorAgeMilliseconds (line, 1min, 1h)
  Widget 1.3: AWS/States ExecutionsStarted/Succeeded/Failed (stacked, 5min)

ROW 2 — IS IT CORRECT?
  Widget 2.1: low_confidence_count (EMF custom; line, 5min)
  Widget 2.2: AWS/States ExecutionsFailed broken by error type (table)
  Widget 2.3: ASL Catch routes — count of fallback executions (EMF custom)

ROW 3 — IS IT FAST?
  Widget 3.1: AWS/Lambda Duration p50/p99 (per function, line, 5min)
  Widget 3.2: AWS/Bedrock InvocationLatency p50/p99 (line, 5min)
  Widget 3.3: AWS/SageMaker ModelLatency + ModelSetupTime (overlay, 5min)
    NOTE: D9 endpoint was deleted at D9 wrap —
  Widget 3.3 shows
    HISTORICAL data only. Re-deploy a temp endpoint
  in Lab 2 if
    you want live SageMaker data; remember to delete
  after.

ROW 4 — IS IT CHEAP?
  Widget 4.1: AWS/Bedrock InputTokenCount + OutputTokenCount (sum, 1h, 7d)
  Widget 4.2: AWS/SageMaker Invocations × variant memory (estimate, 1h)
  Widget 4.3: AWS/Lambda Invocations × Duration (cost proxy, 1h)

```

JSON SHAPE — ONE WIDGET

Cascade scaffolds the rest from this template.

```

{
  "type": "metric", "x": 0, "y": 0, "width":
  12, "height": 6,
  "properties": {
    "metrics": [
      ["AWS/Kinesis", "IteratorAgeMilliseconds",
        "StreamName", "accelya-stream-<learner>",
        {"stat": "Maximum", "period": 60, "label": "max"}],
      [ "...", ".", ".", ".", ".", {"stat": "Average", "label": "avg"} ],
      [
        "region": "ap-south-1",
        "yAxis": { "left": { "min": 0, "label": "ms" } },
        "annotations":
        { "horizontal": [ { "value": 30000, "label": "alarm threshold" } ] },
        "title": "Kinesis IteratorAge — alarm at 30s"
      ]
    ]
  }
}

```

DEPLOY

Single CLI. Re-run to update.

```

aws cloudwatch put-dashboard \
  --dashboard-name accelya-pipeline-$LEARNER \
  --dashboard-body file://infra/dashboard.json \
  --region ap-south-1

# Open the URL it returns:
echo "https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards:name=accelya-pipeline-$LEARNER"

```


Audit accelya-pipeline-role-<learner> end of D9. Replace wildcards. Pair-review MR before apply.

AUDIT — what to look for in the existing role

Three classes of finding:

```
1. Action wildcards:  s3:* dynamodb:*
lambda:* bedrock:* sagemaker:*
2. Resource wildcards: Resource: "*"
3. Missing Conditions: no s3:prefix, no
aws:SecureTransport, no aws:RequestedRegion

## Run this to dump the current role:
ROLE=accelya-pipeline-role-$LEARNER
for POL in $(aws iam list-attached-role-
policies --role-name $ROLE \
--query
'AttachedPolicies[].PolicyArn' --output text);
do
    echo "=== $POL ==="
    VERSION=$(aws iam get-policy --policy-arn
$POL --query 'Policy.DefaultVersionId' --output
text)
    aws iam get-policy-version --policy-arn $POL
--version-id $VERSION \
--query 'PolicyVersion.Document'
done
```

TIGHTENING TABLE

Each row is one MR commit; pair-reviewed before apply.

Action	Before	After
S3 read/write s3:PutObject + s3:ListBucket	s3:* Resource:*	s3:GetObject + Resource: + Condition: s3:prefix +
enriched/<learner>/* quarantine/<learner>/*		
aws:SecureTransport=true Step Functions states:* states:DescribeExecution		states:StartExecution +
arn:...:stateMachine:inference-pipeline-<learner> Bedrock bedrock:* bedrock:InvokeModelWithResponseStream	Resource:*	Resource: bedrock:InvokeModel +
arn:...:inference-profile/apac.anthropic.claude-3-5-sonnet-...	Resource:*	Resource:
SageMaker sagemaker:* sagemaker:InvokeEndpoint		
arn:...:endpoint/accelya-cancel-risk-<learner> Kinesis kinesis:* kinesis:GetShardIterator + DescribeStream	Resource:*	Resource: kinesis:GetRecords +
arn:...:stream/accelya-stream-<learner> DynamoDB dynamodb:* GetItem + UpdateItem		dynamodb:PutItem +
arn:...:table/stream-seen-keys	Resource:*	Resource:

VERIFY

Access Analyzer validates BEFORE apply.

```
aws accessanalyzer validate-policy \
--policy-document file://infra/iam/pipeline-
role-tightened.json \
--policy-type IDENTITY_POLICY \
--region ap-south-1 \
--query 'findings[?findingType==`ERROR` ||
findingType==`SECURITY_WARNING`]'
# Expected: [] - non-empty → STOP, fix, re-
validate.
```

One SNS topic per learner. 4 alarms wired to it. Trainer subscribed for grading.

ONE SNS TOPIC

All alarms publish here.

```
aws sns create-topic --name accelya-ops-
$LEARNER --region ap-south-1
TOPIC_ARN=arn:aws:sns:ap-south-1:$ACCT:accelya-
ops-$LEARNER
# Trainer subscribes; learner subscribes own
email
aws sns subscribe --topic-arn $TOPIC_ARN \
  --protocol email --notification-endpoint
$YOUR_EMAIL --region ap-south-1
```

ALARM 1 — KINESIS LAG (FLOWING?)

Consumer can't keep up.

```
aws cloudwatch put-metric-alarm \
  --alarm-name accelya-kinesis-lag-$LEARNER \
  --metric-name IteratorAgeMilliseconds \
  --namespace AWS/Kinesis \
  --dimensions Name=StreamName,Value=accelya-
stream-$LEARNER \
  --statistic Maximum --period 60 --evaluation-
periods 3 \
  --threshold 30000 --comparison-operator
GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions $TOPIC_ARN --region ap-south-1
```

ALARM 2 — BEDROCK THROTTLE (CORRECT?)

Cap pressure or noisy retry storm.

```
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-
v2:0
aws cloudwatch put-metric-alarm \
  --alarm-name accelya-bedrock-throttle-$LEARNER \
  --metric-name InvocationThrottles --namespace
AWS/Bedrock \
  --dimensions Name=ModelId,Value=$PROFILE \
  --statistic Sum --period 300 --evaluation-periods
1 \
  --threshold 0 --comparison-operator
GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions $TOPIC_ARN --region ap-south-1
```

All 4 alarms live in CloudWatch (real-time, ap-south-1, free per alarm). Cost Explorer + Cost Anomaly Detection are taught as the production pattern but the lab uses CW metric proxies because the Pluralsight sandbox does not enable the Cost Explorer API.

ALL 4 ALARMS IN CLOUDWATCH (this sandbox)

Real-time, region-local, free per alarm. No Cost Explorer dependency.

```
CloudWatch alarms (real-time, ap-south-1, free
per alarm):
  1. Kinesis IteratorAge > 30s
  2. AWS/Bedrock InvocationThrottles > 0/5min
  3. Confidence drift via metric math (above)
  4. CostPerRecordEstimate EMF custom metric >
  $0.001/record sustained 1h
  (proxy alarm – see EMF formula below)

## DASHBOARD COST PROXIES (live, free):
##   - AWS/Bedrock InputTokenCount +
  OutputTokenCount (sum 1h)
##   - AWS/Lambda Duration × Invocations (per
  function)
##   - AWS/Athena DataScannedInBytes (D6
  inheritance, if used)
##   - cost_per_record_estimate EMF metric:
  tokens × $/1K + Lambda-ms × $/100ms
```

PRODUCTION ALTERNATIVE — CE + Cost Anomaly Detection (TAUGHT, NOT RUN)

Reference for production deployments where CE API is enabled. NOT executed in this sandbox.

```
# In a production AWS account WITH Cost
Explorer API enabled, the production
# pattern adds AWS Cost Anomaly Detection as a
5th cost signal:
#   aws ce create-anomaly-monitor --anomaly-
monitor '{...}' --region us-east-1
# Cost data lags 8-24h; ce API endpoint is us-
east-1 only; each
# ce:GetCostAndUsage call is billed (~$0.01).
# In the Pluralsight sandbox: CE API is not
enabled. Use the CW proxies above
# instead. Same cost story, in-region, real-
time, free.
```

BUILD COST_PER_RECORD_ESTIMATE EMF METRIC

Pure-EMF, real-time, learner-controlled.

```
## In merge Lambda (after both branches
return):
BEDROCK_PER_1K_IN  = 0.003    # apac claude 3.5
sonnet
BEDROCK_PER_1K_OUT = 0.015
LAMBDA_PER_100MS   = 0.0000001667 / 1024 * 512
# 512MB billing
tokens_in  =
event['parallel'][0]['usage']['inputTokenCount'
]
tokens_out =
event['parallel'][0]['usage']['outputTokenCount'
]
lambda_ms  =
event['parallel'][1]['model_latency_ms']
cost = (tokens_in/1000)*BEDROCK_PER_1K_IN + \
        (tokens_out/1000)*BEDROCK_PER_1K_OUT + \
        (lambda_ms/100)*LAMBDA_PER_100MS
metrics.add_metric(name='CostPerRecordEstimate'
,
                    unit=MetricUnit.None_,
                    value=cost)
## Lab target: <$0.001/record over 7d (no SM
endpoint left running).
```

Tag every resource. Per-tag cost report. Compute cost-per-record.

TAG EVERY RESOURCE

3 tags mandatory; Cascade scaffolds with these baked in.

```
Tags (3 mandatory):
env=lab
learner=$LEARNER
module=10

## Add tags retroactively to all 10 Lambdas:
for FN in $(aws lambda list-functions --region
ap-south-1 \
--query
"Functions[?contains(FunctionName,'$LEARNER')].
FunctionArn" --output text); do
aws lambda tag-resource --resource $FN \
--tags env=lab,learner=$LEARNER,module=10 -
-region ap-south-1
done

## SFN, Kinesis, DynamoDB, SageMaker — same
pattern.
## PRODUCTION NOTE: cost-allocation tags must
be ACTIVATED in Billing Console
## once per account; takes ~24h to appear in
Cost Explorer. NOT applicable in
## this sandbox — CE API is not enabled here.
Tags still drive CW dimensioning.
```

PER-TAG COST REPORT — CW PROXY (sandbox path)

CW metric-math derives per-learner cost from publicly-available unit prices + actual usage. No CE API required.

```
## CW Insights — per-tag service usage (sandbox-runnable):
## (the dimension is the EMF Function/Service tag, not a CE
Tag)
stats sum(InvocationCount) as inv,
sum(InputTokenCount) as in_tok,
sum(OutputTokenCount) as out_tok,
sum(DataScannedInBytes) as athena_bytes,
sum(Duration) as lambda_ms
by bin(1d)
from AWS/Bedrock, AWS/Lambda, AWS/Athena
filter learner = '$LEARNER'

## CostPerRecordEstimate EMF metric (the merge Lambda emits
this):
## cost = (in_tok/1000)*$0.003 + (out_tok/1000)*$0.015
## + (lambda_ms/100)*$0.000001667/MB*MB
## + (athena_bytes/1e12)*$5

## Expected breakdown after a busy lab day (proxy estimate,
not invoice):
## AWS Lambda ~$0.04
## Amazon Kinesis Data Streams ~$0.12
## Amazon Bedrock ~$0.18
## Amazon SageMaker ~$0.45 (if endpoint not
deleted!)
## CloudWatch ~$0.06
## TOTAL (estimate) ~$0.85

## PRODUCTION ALTERNATIVE (taught only):
## aws ce get-cost-and-usage --filter
'{"Tags":{"Key":"learner",...}}'
## --region us-east-1 # CE endpoint is us-east-1 ONLY
```

COST-PER-RECORD

The metric that turns observability into a P&L conversation.

```
## Cost-per-record = total $ / records
processed in window
## EMF emits
records_processed_per_correlation_id; sum
across window.
## CW Insights derives:
stats sum(records_processed) as total_records
by bin(1d)
## Combine with the CW-derived cost above →
cost / total_records.
## Lab target: cost-per-record < $0.001 over a
7-day window
## (without SageMaker endpoint left running;
with it, expect ~$0.01).
```

Designs for dashboard + IAM + alarms + cost

Dashboard JSON answering 4 questions. IAM tightening table mapping wildcards → scoped. FOUR CW alarms (Kinesis lag + Bedrock throttle + confidence drift + cost-per-record proxy). Tags + cost-per-record EMF metric. Cost Explorer taught conceptually; not used in this sandbox.

DESIGNED

1 dashboard, 11 widgets, 4 rows. 1 IAM tightening MR with 7 commits. 4 CW alarms on 1 SNS topic. Tag scheme (env+learner+module).

WHAT CASCADE WILL SCAFFOLD

powertools Logger + Metrics decorators on each of 10 Lambdas. Dashboard JSON. Tightened IAM policy. CostPerRecordEstimate EMF metric. Alarm CLI block.

WHAT'S NEXT

Three labs: STAGED powertools rollout (Lab 1, 45m), dashboard + EMF + 4 CW alarms (Lab 2, 45m), IAM MR with safe rollout + cost baseline via CW proxies (Lab 3, 45m).

03

PART 3 · ~135 MINUTES (3 × 45)

Three labs

Lab 1 — powertools Logger + correlation IDs across all 10 Lambdas + verify X-Ray. Lab 2 — dashboard JSON + EMF metrics + 4 SNS alarms. Lab 3 — IAM tightening MR (6 commits) + cost baseline document.

powertools rollout + correlation IDs + X-Ray verification

BRIEF

Pair-work. Cascade-author `src/common/observability.py` (Logger/Metrics/Tracer factories per OBSERVABILITY CONTEXT layer). Roll out across all 10 Lambdas: 3 decorators each. Producer assigns `correlation_id = f'corr-{uuid4()}'` INTO envelope. Consumer extracts via `correlation_id_path='envelope.correlation_id'`; downstream Lambdas extract from top-level `'correlation_id'`. Enable X-Ray Active on all 9 + SFN; attach `AWSXRayDaemonWriteAccess` to role. `patch_all()` at module level (NOT in handler — won't apply on warm).

TIME

45 min

LEVEL

Working+

DELIVERABLE

`src/common/observability.py` + 9 instrumented handlers + 1 producer-with-corr-id + `xray-sampling-rule.json` applied + `reviews/m10-xray-trace-evidence.md` (screenshot of one cross-service trace) + MR commit "module-10: powertools rollout (Lab 1)"

powertools + correlation IDs + X-Ray verify

1

Cascade-author shared observability module

```
# Open Cascade with .windsurfules v8 (with OBSERVABILITY CONTEXT)
# Prompt: 'Create src/common/observability.py exporting get_logger, get_metrics,
#         get_tracer factories per OBSERVABILITY CONTEXT layer. Namespace
#         AccelyaPipeline/STARNR. Auto-include correlation_id path.'
```

STAGED rollout — common module → 3 representative Lambdas → smoke → remaining 6

2

```
# Stage A: common module ONLY (already done in step 1)
# Stage B: 3 representative Lambdas (producer = upstream / consumer = SFN-handoff /
#         predict = downstream callee)
# Cascade prompts (one per file):
# 'Apply powertools decorators to src/produce/handler.py per OBSERVABILITY
#   CONTEXT layer. Preserve existing logic.' (repeat for consumer + predict)
# Smoke 1 record end-to-end; verify CW Insights query for the corr-id returns
# log lines from the 3 instrumented + raw lines from the 6 not-yet-instrumented.
```

3

Producer assigns correlation_id (with fallback chain)

```
# Edit src/produce/handler.py — single change in envelope construction.
# In src/common/observability.py — fallback extractor:
def get_correlation_id(event: dict) -> str:
    return (event.get('envelope', {}).get('correlation_id')
            or event.get('correlation_id')
            or f'corr-{uuid.uuid4()}') # last-resort fallback (and log warning)
# Use in every handler so mixed-source events always have a corr-id.
```

Enable X-Ray on all 9 + SFN; SFN role gets PutTraceSegments + GetSamplingRules

4

```
for FN in $(aws lambda list-functions --region ap-south-1 \
--query "Functions[?contains(FunctionName, '$LEARNER')].FunctionName" --output text); do
aws lambda update-function-configuration \
--function-name $FN --tracing-config Mode=Active --region ap-south-1
done
aws stepfunctions update-state-machine --state-machine-arn $SM_ARN \
--tracing-configuration enabled=true --region ap-south-1

# Lambda role — managed policy is enough:
aws iam attach-role-policy --role-name accelya-pipeline-role-$LEARNER \
--policy-arn arn:aws:iam::aws:policy/AWSXRayDaemonWriteAccess
```

5

Pass traceHeader on StartExecution (otherwise SFN starts a NEW root trace)

```
# Edit src/consumer/handler.py — wrap the StartExecution call:
from aws_xray_sdk.core import xray_recorder
seg = xray_recorder.current_segment()
trace_header = f'Root={seg.trace_id};Parent={seg.id};Sampled=1'
sfn.start_execution(stateMachineArn=SM_ARN,
                    name=f'inf-{event_id}',
                    input=json.dumps(payload),
                    traceHeader=trace_header) # stitches consumer trace into SFN
# Without traceHeader, X-Ray records execution as a disconnected trace.
```

6

Send 1 test record + verify trace

```
aws lambda invoke --function-name accelya-producer-$LEARNER \
--payload file:///tmp/m10-1.json /tmp/m10-1-out.json --region ap-south-1
sleep 60 # let trace materialise
# Open: console.aws.amazon.com/xray/home?region=ap-south-1#/service-map
# Capture screenshot → reviews/m10-xray-trace-evidence.md
```

EXPECTED

src/common/observability.py with Logger/Metrics/Tracer factories.

EXPECTED

common.observability module landed; 3 reps (producer + consumer + wrapper) decorated; smoke 1 record passes BEFORE rolling out the other 6.

EXPECTED

src/produce/handler.py emits envelope.correlation_id = f'corr-{uuid4()}' per record. Fallback chain documented in src/common/observability.py.

EXPECTED

All Lambdas TracingConfig=Active; SFN tracingConfiguration.enabled=true; TWO different X-Ray policies on TWO different roles.

EXPECTED

consumer/handler.py builds traceHeader from current X-Ray segment + passes to start_execution.

EXPECTED

X-Ray service map shows Producer → Kinesis → Consumer → SFN → Bedrock + Wrapper → SageMaker → Merge.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- powertools imported in all 9 handlers; decorators applied
- X-Ray Active on all 10 Lambdas + SFN; AWSXRayDaemonWriteAccess on role
- ColdStart=1 metric visible under AccelyaPipeline/<learner>
- X-Ray service map shows ≥5 segments in one trace
- CW Insights query returns ≥4 log groups for one correlation_id
- MR commit: module-10: powertools rollout (Lab 1) pushed

⚠ TOP GOTCHAS

Forgot AWSXRayDaemonWriteAccess on role

Lambda has Active tracing but role lacks permission → traces silently dropped. Attach policy.

patch_all() called in handler instead of import time

Patches don't apply on warm invokes. Move to module level.

correlation_id_path mismatch between consumer + downstream

Consumer reads envelope.correlation_id; downstream reads correlation_id. Both must match what handler injects.

log_event=True logs PII

Default decorator flag dumps the full event. Set log_event=False for any handler that sees passenger data.

AccelyaPipeline namespace already exists from old test

Old metrics may pollute dashboard. Use AccelyaPipeline/\$LEARNER (per-learner suffix).

Dashboard JSON + EMF custom metrics + 4 SNS alarms

BRIEF

Pair-work. Cascade-author infra/dashboard.json (4 rows × 2-3 widgets) per Guided Design. Wire 4 EMF metrics in wrapper Lambda: ModelSetupMs, LowConfidenceCount, ThrottleRetries, ConfidenceValue. Create 1 SNS topic accelya-ops-<learner>; trainer + learner subscribed (confirmed). Apply 4 alarms: Kinesis IteratorAge>30s, AWS/Bedrock InvocationThrottles>0/5min, confidence drift >20% in 1h, daily cost >\$5/learner. Smoke-test one alarm via set-alarm-state → email arrives.

TIME

45 min

LEVEL

Working+

DELIVERABLE

infra/dashboard.json + 4 EMF emissions + 1 SNS topic with confirmed subs + 4 CW alarms + reviews/m10-dashboard-screenshot.md + MR commit "module-10: dashboard + alarms (Lab 2)"

Dashboard + EMF + alarms

1

Cascade-author dashboard JSON

```
# Cascade prompt: 'Create CloudWatch dashboard JSON per Guided Design slide.
# 4 rows x 2-3 widgets. Per OBSERVABILITY CONTEXT - namespaces
# AWS/Kinesis, AWS/Lambda, AWS/Bedrock, AWS/SageMaker, AWS/States,
# AccelyaPipeline/$LEARNER. Region ap-south-1.'
# Output ~250 lines JSON. Review widget IDs + dimensions.
```

EXPECTED

infra/dashboard.json with 4 rows × widgets per Guided Design slide.

2

Add EMF emissions to wrapper Lambda

```
# Cascade prompt: 'Edit src/predict/handler.py to emit 4 EMF metrics per
# OBSERVABILITY CONTEXT custom-metrics list. Use existing metrics object.'
# Re-package + update:
(cd src/predict && zip -r ../../predict.zip . && cd ../../)
aws lambda update-function-code --function-name accelya-predict-cancel-risk-$LEARNER \
  --zip-file fileb://predict.zip --region ap-south-1
```

EXPECTED

src/predict/handler.py emits 4 metrics: ModelSetupMs, LowConfidenceCount, ThrottleRetries, ConfidenceValue.

3

Deploy dashboard

```
aws cloudwatch put-dashboard \
  --dashboard-name accelya-pipeline-$LEARNER \
  --dashboard-body file://infra/dashboard.json \
  --region ap-south-1
echo "https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-
```

EXPECTED

Dashboard URL returns; widgets render.

4

Create SNS topic + subscribe

```
aws sns create-topic --name accelya-ops-$LEARNER --region ap-south-1
TOPIC_ARN=$(aws sns list-topics --region ap-south-1 \
  --query "Topics[?contains(TopicArn,'accelya-ops-$LEARNER')].TopicArn | [0]" --output text)
aws sns subscribe --topic-arn $TOPIC_ARN \
  --protocol email --notification-endpoint $YOUR_EMAIL --region ap-south-1
aws sns subscribe --topic-arn $TOPIC_ARN \
  --protocol email --notification-endpoint trainer@accelya.example --region ap-south-1
```

EXPECTED

Topic ARN exported; trainer + learner email subscribed (confirmed).

5

Apply 4 CW alarms (all in CloudWatch ap-south-1; CE/CAD not used in this sandbox)

```
TOPIC_ARN=arn:aws:sns:ap-south-1:$ACCT:accelya-ops-$LEARNER
bash infra/alarms/kinesis-lag.sh
bash infra/alarms/bedrock-throttle.sh
bash infra/alarms/confidence-drift.sh
bash infra/alarms/cost-per-record.sh # alarms on the EMF CostPerRecordEstimate metric
# Cost Explorer / Cost Anomaly Detection are taught in Guided Design as the
# production pattern but NOT used here - Pluralsight sandbox does not enable
# the ce:* API. All cost evidence comes from CW metric proxies in-region.

aws cloudwatch describe-alarms --region ap-south-1 \
  --alarm-names accelya-kinesis-lag-$LEARNER \
    accelya-bedrock-throttle-$LEARNER \
    accelya-confidence-drift-$LEARNER \
    accelya-cost-per-record-$LEARNER \
  --query 'MetricAlarms[].[Name:AlarmName,State:StateValue]'
```

EXPECTED

4 CloudWatch alarms exist in ap-south-1; all wired to the SNS topic.

Dashboard + EMF + alarms (cont.)

6

Smoke-test alarms — TWO named checks (do NOT force a Bedrock throttle)

```
## CHECK 1 — "SNS WIRING TEST via set-alarm-state"
## Proves: alarm action → SNS → email pipeline works.
## NOTE: set-alarm-state is TEMPORARY — metric alarms return to actual state
## quickly (next evaluation). Use it for SNS proof only.
aws cloudwatch set-alarm-state --alarm-name accelya-kinesis-lag-$LEARNER \
  --state-value ALARM --state-reason 'lab SNS path test' --region ap-south-1
# Email arrives within ~30s. Alarm self-resets on next eval; nothing to do.

## CHECK 2 — "METRIC-DIMENSION TEST via low-threshold existing metric"
## DO NOT try to force a Bedrock throttle by invoking a non-allowed model —
## that produces AccessDeniedException / ValidationException, which does
## NOT increment AWS/Bedrock InvocationThrottles. The alarm wouldn't fire
## even if wired correctly, and the test gives a false-negative.
## Instead: temporarily lower the threshold against an EXISTING, already-
## populated metric (Bedrock Invocations from D7-D9 runs):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
# Save current threshold:
ORIG=$(aws cloudwatch describe-alarms --region ap-south-1 \
  --alarm-names accelya-bedrock-throttle-$LEARNER \
  --query 'MetricAlarms[0].Threshold' --output text)
echo "ORIG_THRESHOLD=$ORIG" > /tmp/m10-alarm-rollback.env

# Create a TEMP alarm against AWS/Bedrock Invocations with threshold > -1
# (so any historical invocation breaches it; uses metric/dimension exactly
# like the real throttle alarm would, just on a populated metric):
aws cloudwatch put-metric-alarm \
  --alarm-name accelya-metric-dim-test-$LEARNER \
  --metric-name Invocations --namespace AWS/Bedrock \
  --dimensions Name=ModelId,Value=$PROFILE \
  --statistic Sum --period 300 --evaluation-periods 1 \
  --threshold -1 --comparison-operator GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions $TOPIC_ARN --region ap-south-1
# Within 5min, alarm transitions to ALARM from REAL metric data → SNS fires.
# This proves metric/dimension wiring (namespace + dim name + stat all valid).

# Cleanup the temp alarm:
aws cloudwatch delete-alarms --region ap-south-1 \
  --alarm-names accelya-metric-dim-test-$LEARNER

# Document both events (set-alarm-state ALARM + temp-alarm ALARM)
# in reviews/m10-dashboard-screenshot.md so trainer sees both checks.
# Capture dashboard screenshot.
```

EXPECTED

(1) SNS wiring proven via set-alarm-state on any alarm; (2) metric/dimension wiring proven via a temporary low-threshold breach on an EXISTING metric (Bedrock Invocations or InputTokenCount).

Success criteria & common gotchas

✓ YOU ARE READY IF...

- infra/dashboard.json deployed; URL renders all 4 rows
- EMF metrics ModelSetupMs, LowConfidenceCount, ThrottleRetries, ConfidenceValue, CostPerRecordEstimate visible
- 1 SNS topic + 2 subscriptions confirmed
- 3 CW alarms exist + cost monitor verified separately (trainer-provisioned)
- Smoke-tests: BOTH set-alarm-state (SNS path) AND one real-threshold breach (metric correctness)
- /reviews/m10-dashboard-screenshot.md committed

⚠ TOP GOTCHAS

Dashboard widget shows 'No data'

Wrong namespace or dimension. Re-check namespace = AccelyaPipeline/\$LEARNER (not AccelyaPipeline).

EMF metric not appearing

Lambda must have logged at least once with that metric name. Send a test record.

SNS subscription pending

Click confirm link in email; alarms can publish but no email goes out until confirmed.

Confidence drift always INSUFFICIENT_DATA

Needs ≥1h of EMF emissions + ≥10 datapoints/hour (count-guard). Skip until Lab 3 if pressed for time; document as known.

set-alarm-state ≠ metric correctness

Proves SNS wiring only. ALSO do one real-threshold test (force a Bedrock throttle) to prove metric/dimension wiring.

IAM tightening MR + cost baseline document

BRIEF

Solo. Dump accelya-pipeline-role-<learner> end of D9. Cascade-author infra/iam/AUDIT_FINDINGS.md (3 wildcard classes + line refs). Cascade-author infra/iam/pipeline-role-tightened.json per Guided Design tightening table (S3 + SFN + Bedrock + SageMaker + Kinesis + DynamoDB). Validate via Access Analyzer (expect [] findings). Open MR with 7 commits (1 audit + 6 service tightenings); pair partner tagged. After pair review: create-policy-version --set-as-default; smoke-test 1 record (must SUCCEED). Capture cost baseline: top 3 services + cost-per-record + 3 optimisations to /reviews/m10-cost-baseline.md.

TIME

45 min

LEVEL

Working+

DELIVERABLE

infra/iam/AUDIT_FINDINGS.md +
infra/iam/pipeline-role-tightened.json (validate
clean) + GitLab MR with 7 commits + applied policy
version + 1 successful smoke-test execution +
reviews/m10-cost-baseline.md

IAM MR + cost baseline

1

Dump current role + identify wildcards

```

ROLE=accelya-pipeline-role-$LEARNER
for POL in $(aws iam list-attached-role-policies --role-name $ROLE \
  --query 'AttachedPolicies[].PolicyArn' --output text); do
  echo "=== $POL ==="
  V=$(aws iam get-policy --policy-arn $POL --query 'Policy.DefaultVersionId' --output text)
  aws iam get-policy-version --policy-arn $POL --version-id $V \
    --query 'PolicyVersion.Document'
done > infra/iam/role-current.json

# Cascade prompt: 'Audit infra/iam/role-current.json. List every Action wildcard,
# Resource wildcard, missing Condition. Output as infra/iam/AUDIT_FINDINGS.md.'
```

EXPECTED

AUDIT_FINDINGS.md lists 3 wildcard classes with line references.

2

Cascade-author tightened policy

```

# Cascade prompt: 'Apply the tightening table from Guided Design slide:
# - S3: explicit verbs + Resource arn:../enriched/$LEARNER/* + s3:prefix Condition
# - SFN: scope to inference-pipeline-$LEARNER
# - Bedrock: scope to apac inference profile
# - SageMaker: scope to accelya-cancel-risk-$LEARNER endpoint ARN
# - Kinesis: GetRecords/GetShardIterator/DescribeStream + stream ARN
# - DynamoDB: PutItem/GetItem/UpdateItem + table ARN
# Output infra/iam/pipeline-role-tightened.json'
```

EXPECTED

infra/iam/pipeline-role-tightened.json per Guided Design tightening table.

IAM MR + cost baseline (cont.)

3

Validate with Access Analyzer (gate on ERROR + SECURITY_WARNING ONLY) — STATIC check, NOT runtime

```
# BLOCKING gate — STOP if any ERROR or SECURITY_WARNING:
BLOCKERS=$(aws accessanalyzer validate-policy \
  --policy-document file://infra/iam/pipeline-role-tightened.json \
  --policy-type IDENTITY_POLICY --region ap-south-1 \
  --query 'findings[?findingType==`ERROR` || findingType==`SECURITY_WARNING`]' \
  --output json)
[ "$BLOCKERS" = "[]" ] || { echo "BLOCKERS:"; echo "$BLOCKERS"; exit 1; }

# NON-BLOCKING — log for review but proceed:
aws accessanalyzer validate-policy \
  --policy-document file://infra/iam/pipeline-role-tightened.json \
  --policy-type IDENTITY_POLICY --region ap-south-1 \
  --query 'findings[?findingType==`GENERAL_WARNING` || findingType==`SUGGESTION`]' \
  > infra/iam/access-analyzer-non-blocking.json
# Review file in MR; commit alongside tightened policy.
```

EXPECTED

No ERROR or SECURITY_WARNING findings. GENERAL_WARNING / SUGGESTION logged separately for review (not blockers). NOTE: Access Analyzer is a STATIC validator — it cannot prove runtime permissions. The canary smoke in step 4 is the true runtime gate.

4

Open MR — 6 reviewable commits

```
git checkout -b module-10-iam-tightening-$LEARNER
git add infra/iam/AUDIT_FINDINGS.md
git commit -m "module-10: audit findings (3 wildcard classes)"
# 6 service commits — split the policy edits manually:
git add infra/iam/pipeline-role-s3.json && git commit -m "module-10: tighten S3 (explicit verbs + prefix + SecureTransport)"
git add infra/iam/pipeline-role-sfn.json && git commit -m "module-10: tighten SFN (single ARN)"
git add infra/iam/pipeline-role-bedrock.json && git commit -m "module-10: tighten Bedrock (apac profile only)"
git add infra/iam/pipeline-role-sagemaker.json && git commit -m "module-10: tighten SageMaker (single endpoint)"
git add infra/iam/pipeline-role-kinesis.json && git commit -m "module-10: tighten Kinesis (single stream + read verbs)"
git add infra/iam/pipeline-role-dynamodb.json && git commit -m "module-10: tighten DynamoDB (single table + 3 verbs)"
git push -u origin module-10-iam-tightening-$LEARNER
# Open MR via GitLab UI; tag pair partner.
```

EXPECTED

GitLab MR shows 6 commits one-per-service; pair partner tagged.

IAM MR + cost baseline (cont.)

5

SAFE ROLLOUT — capture prev version, prune if at cap, apply, canary, rollback on fail

```
# IAM managed policies cap at 5 versions. Prune oldest non-default FIRST
# so create-policy-version doesn't fail with LimitExceeded.
PREV_VERSION=$(aws iam get-policy --policy-arn $POLICY_ARN \
  --query 'Policy.DefaultVersionId' --output text)
echo "PREV_VERSION=$PREV_VERSION" > /tmp/m10-rollback.env # SAVED for rollback

# Prune oldest non-default if at the 5-version cap:
VERSIONS=$(aws iam list-policy-versions --policy-arn $POLICY_ARN \
  --query 'Versions | sort_by(@, &CreateDate)' --output json)
VCOUNT=$(echo "$VERSIONS" | jq 'length')
if [ "$VCOUNT" -ge 5 ]; then
  OLDEST=$(echo "$VERSIONS" | jq -r '[] | select(.IsDefaultVersion==false) | .VersionId' | head -1)
  aws iam delete-policy-version --policy-arn $POLICY_ARN --version-id $OLDEST
fi
```

EXPECTED

Previous version ID saved; one-record canary SUCCEEDED; rollback path proven (dry).

6

Capture cost baseline (CW proxy — sandbox path; CE call shown as production reference)

```
## CW PROXY (sandbox-runnable):
## Pull token + duration + scanned-bytes per service, last 7 days, in-region.
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in InputTokenCount OutputTokenCount Invocations; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d '7 days ago' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 86400 --statistics Sum \
    --region ap-south-1
done > /tmp/cost-bedrock-7d.json

for FN in $(aws lambda list-functions --region ap-south-1 \
  --query "Functions[?contains(FunctionName,'$LEARNER')].FunctionName" --output text); do
  aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
    --metric-name Duration --dimensions Name=FunctionName,Value=$FN \
    --start-time $(date -u -d '7 days ago' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 86400 --statistics Sum \
    --region ap-south-1
done > /tmp/cost-lambda-7d.json
```

EXPECTED

/reviews/m10-cost-baseline.md with top 3 services + cost-per-record + 3 optimisations.

IAM MR + cost baseline (cont.)

6

Capture cost baseline (CW proxy — sandbox path; CE call shown as production reference)

```
# Cascade prompt: 'Read /tmp/cost-bedrock-7d.json + /tmp/cost-lambda-7d.json +
# AccelyaPipeline/$LEARNER records_processed sum. Apply published unit prices
# ($0.003/1K input + $0.015/1K output for apac claude 3.5 sonnet;
# $0.0000001667/100ms-MB for Lambda). Compute estimated cost + cost-per-record.
# Identify top 3 cost drivers + 3 optimisations.
# Write reviews/m10-cost-baseline.md.'
git add reviews/m10-cost-baseline.md
git commit -m "module-10: cost baseline via CW proxy (Lab 3)"
git push

## PRODUCTION REFERENCE (CE call — NOT runnable here; CE API not enabled):
##   aws ce get-cost-and-usage \
##     --time-period Start=$(date -u -d '7 days ago' +%F),End=$(date -u +%F) \
##     --granularity DAILY --metrics BlendedCost \
##     --group-by Type=DIMENSION,Key=SERVICE \
##     --filter '{"Tags":{"Key":"learner","Values":["'$LEARNER'"]}}' \
##     --region us-east-1 # CE endpoint is us-east-1 only
```

EXPECTED

/reviews/m10-cost-baseline.md with top 3 services + cost-per-record + 3 optimisations.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- MR with 8 commits (1 audit + 6 service tightenings + 1 access-analyzer-non-blocking.json) + pair-reviewed
- PREV_VERSION saved to /tmp/m10-rollback.env BEFORE create-policy-version
- Tightened policy applied as new default; canary 1 record SUCCEEDED (or auto-rolled-back)
- Access Analyzer ERROR + SECURITY_WARNING gate = [] (non-blocking findings reviewed in MR)
- /reviews/m10-cost-baseline.md captures top 3 services + cost-per-record + 3 optimisations
- Lambda count cumulative still 10 of 10 (no new functions today)

⚠ TOP GOTCHAS

Forgetting to set --set-as-default

create-policy-version creates a new version but doesn't activate it. Always pass --set-as-default.

5-version cap on managed policies — LimitExceeded

Prune oldest non-default version BEFORE create. Lab playbook does this in step 4.

Skipping the canary smoke

Access Analyzer is STATIC. Only a real execution proves the tightened policy doesn't break runtime. NEVER apply without canary + auto-rollback path.

Forgot to capture PREV_VERSION

If canary fails and you didn't save the prior default version ID, you're rolling back blind. Always echo PREV_VERSION to /tmp/m10-rollback.env first.

Cost Explorer API not enabled in this sandbox

Use CW metric proxies (token counts + Lambda duration + Athena scanned bytes); CE call kept in deck as production reference only.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-10 mistakes. End-of-day cleanup (no endpoint to delete today). Day-10 in numbers. Bridge to Day 11 — End-to-End Integration WITH CI/CD inside. The DEPLOY CONTEXT block in .windsurfrules v8.

Five Day-10 observability mistakes — and what to do instead

01

No correlation_id — debug by tail-ing CloudWatch

MISTAKE When a record disappears between SFN start and merge, you tail 9 log groups manually. By the time you've found anything, the next batch is failing too.

DO INSTEAD

Producer assigns f'corr-{uuid4()}' INTO the envelope. powertools Logger inject_lambda_context picks it up. CW Insights query 'filter correlation_id = X' returns every line for one record.

02

Custom metric where AWS/* covers it

MISTAKE Adding 'lambda_duration' as a custom metric. Already published as AWS/Lambda Duration — you're paying twice and the data isn't comparable across services.

DO INSTEAD

OBSERVABILITY CONTEXT layer enumerates the FORBIDDEN custom metrics. AWS/* always first; custom only for the gap.

03

IAM wildcards persisted to production

MISTAKE s3:* + Resource:* worked in lab and got copy-pasted to prod. SOC 2 finding; data exfiltration risk.

DO INSTEAD

Lab 3 IAM-tightening MR pattern is the production pattern. Access Analyzer validate-policy gates merges. No exceptions.

04

No Bedrock metrics on the dashboard

MISTAKE Bedrock throttling silently retries; you see slowness but not the cause. Eventually hit the cap and the pipeline stalls.

DO INSTEAD

AWS/Bedrock InvocationThrottles widget on dashboard + alarm at >0/5min. First sign of cap pressure visible immediately.

05

Log retention 'never expire'

MISTAKE Default CW Logs retention is unlimited. After 6 months you're paying \$400/mo for logs nobody reads.

DO INSTEAD

Set retention 14d (lab) or 30d (prod) on every log group. Cascade scaffolds with --log-group-class STANDARD --retention-in-days 14.

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-10-<learner>; MR with 8+ commits; pair partner tagged
- Dashboard URL accelya-pipeline-<learner> deployed and screenshot captured
- 3 CW alarms in OK / INSUFFICIENT_DATA + cost monitor verified separately
- IAM policy tightened (ERROR + SECURITY_WARNING gate clean) + canary smoke SUCCEEDED + PREV_VERSION saved
- Lambda count UNCHANGED: 10 of 10 (instrumentation only; cap was reached on D9; D11 uses ALARM-based rollback)
- Bedrock invocations: ~40 of 50 (5 used by Cascade-authoring); 10 left for D11
- All 9 Lambda log groups now have 14d retention (verify list-log-groups)
- X-Ray sampling rule applied at 100% — leave for now; D11 can drop to 10%

```
# Verify retention is set on all log groups:
aws logs describe-log-groups --region ap-south-1 \
  --log-group-name-prefix /aws/lambda/accelya- \
  --query 'logGroups[].[Name:logGroupName,Retention:retentionInDays]'
```

```
aws lambda list-functions --region ap-south-1 \
  --query "length(Functions[?contains(FunctionName,'$LEARNER')])"
```

```
# Must output: 9 (no new Lambdas this module)
```

```
# Bedrock budget (3 metrics) — same query you ran D7/D8/D9:
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 'today 00:00' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 3600 --statistics Sum \
    --region ap-south-1
done
```

```
# D9 endpoint should already be deleted from yesterday — verify:
aws sagemaker list-endpoints --region ap-south-1 \
  --query "length(Endpoints[?contains(EndpointName,'$LEARNER')])"
```

```
# Must output: 0
```

Trainer note: Lambda count 10 of 10 — final cap reached on D9. D11 uses alarm-based rollback (no extra Lambda). Bedrock ~48/50 — D11 needs ~2 (CI/CD authoring); D12 reserves ~2 for capstone. All 9 functions now powertools-instrumented; D11 SAM templates inherit observability hooks for free.

What you actually shipped

0

NEW LAMBDA

Instrumentation only. Cumulative 10 of 10 (cap was reached on D9).

1

DASHBOARD

4 rows. 11 widgets. Answers flowing/correct/fast/cheap.

3 + 1

ALARMS

3 CW alarms (Kinesis lag, Bedrock throttle, confidence drift) + 1 cost monitor (CE us-east-1, lagged).

13

PROMPT LAYERS

+ OBSERVABILITY CONTEXT. Cascade scaffolds with hooks built-in.

Day 11 — End-to-End Integration with CI/CD

Today the pipeline became observable. Tomorrow it becomes deployable: AWS SAM templates + GitLab CI/CD + OSS scanners (bandit, semgrep, tflint) + OIDC trust between GitLab and AWS — no static keys. The DEPLOY CONTEXT block in .windsurfrules v8 enters.

SAM + GITLAB

All 10 Lambdas + ASL + alarms move into one SAM template (no new Lambdas tomorrow)

Cascade authors template.yaml. GitLab CI runs sam validate + sam build + sam deploy via OIDC role assumption (no AWS keys in GitLab variables).

OSS SCANNERS

bandit (Python SAST) + semgrep (multi-lang) + tflint (template lint)

All three run in CI. Findings break the build. PR-style gates on the IAM policy too (re-validate via Access Analyzer in CI).

SANDBOX BUDGET

+1 Lambda spare + ~5 Bedrock invocations

Cumulative D11: 10 of 10 Lambdas; ~45 of 50 Bedrock used. D12 capstone uses what's left.

Day 11 branch: module-11-<learner>. Inherits .windsurfrules v7 (with OBSERVABILITY CONTEXT). Dashboard, alarms, IAM tightening all preserved.

The system has eyes.

DASHBOARD + ALARMS

1 dashboard answering 4 questions. 4 SNS alarms wired. Cost-per-record measurable.

IAM TIGHTENED

Wildcards → ARN-scoped. Access Analyzer clean. SOC-2 path opened.

.WINDSURFRULES v8

OBSERVABILITY CONTEXT block in .windsurfrules v7. powertools everywhere. AWS/* before custom.

Day 11 starts at 09:00. Bring your 10 Lambdas + dashboard + alarms + tightened IAM. Deploy enters; the DEPLOY CONTEXT block in .windsurfrules v8 joins.